

“Año de la Esperanza y el Fortalecimiento de la Democracia”



UNIVERSIDAD NACIONAL DE LA AMAZONÍA PERUANA
Ingeniería de Sistemas e Informática



SERVIDESK

DOCUMENTO DEL PROYECTO

CURSO

Gestión de Servicios en TI

DOCENTE

Ing. Carlos Gonzalez Aspajo Mtr.

GRUPO

Danny Steve Ordoñez Navarro
Paredes Burga Alexander Ali
Grandez Sifuentes Jhor Anthony
Burga Saldaña André de Jesús
Silva Ramos Ludwing

IQUITOS - LORETO - PERÚ

2026

RESUMEN EJECUTIVO

ServiDesk es una plataforma SaaS de gestión de soporte técnico TI diseñada específicamente para el segmento de empresas medianas en Latinoamérica con áreas de tecnología internas de entre 5 y 100 usuarios. La propuesta nace de una problemática identificada en el mercado: la mayoría de organizaciones de este tamaño gestiona sus incidencias técnicas mediante canales informales como WhatsApp, correo electrónico y comunicación verbal, sin trazabilidad ni indicadores de desempeño, mientras que las soluciones empresariales existentes como Zendesk o Freshdesk resultan costosas y sobredimensionadas para sus necesidades reales.

El producto se construye bajo un modelo freemium escalable con tres planes: Gratis (limitado a 50 tickets/mes), Básico (S/.29/mes con tickets ilimitados y analytics básico) y Pro (S/.79/mes con inteligencia artificial integrada, analytics avanzado y soporte prioritario). La plataforma incluye módulos de gestión de tickets, asignación de técnicos, sistema de evaluación, base de conocimiento con búsqueda, dashboard de analíticas, y clasificación automática mediante IA. Los cobros recurrentes se procesan a través de Culqi, la pasarela de pagos peruana líder en suscripciones digitales.

El desarrollo se ejecuta en un semestre académico de 14 semanas siguiendo la metodología Scrum, distribuida en siete sprints de dos semanas cada uno. La arquitectura técnica está fundamentada en un stack moderno full-stack basado en Next.js 15 con TypeScript, PostgreSQL como base de datos relacional, Clerk como proveedor de autenticación gestionada con soporte multi-tenant nativo, y la API de Google Gemini para los componentes de inteligencia artificial. El despliegue se realiza sobre la infraestructura serverless de Vercel.

El presente documento corresponde al primer entregable del proyecto y cubre la fase de análisis, diseño arquitectónico y diseño de interfaces de usuario, dejando la fase de implementación y ejecución de sprints para los siguientes entregables del semestre.

I. INTRODUCCIÓN

1. Contexto del producto

La gestión del soporte técnico interno representa uno de los procesos operativos más críticos y, paradójicamente, peor sistematizados en las organizaciones medianas. A medida que una empresa crece, la cantidad de incidencias tecnológicas que enfrenta diariamente aumenta proporcionalmente: cuentas de usuario que requieren creación o modificación, equipos que fallan, software que no responde, accesos que deben renovarse, instalaciones de licencias, configuración de impresoras y un largo etcétera de tareas que recaen sobre un área TI generalmente reducida en personal.

En este contexto, la falta de un sistema formal de gestión de tickets genera consecuencias medibles: pérdida de trazabilidad sobre qué incidencias se atendieron, cuándo y por quién; imposibilidad de identificar problemas recurrentes que podrían resolverse de raíz; ausencia de indicadores objetivos sobre el desempeño del equipo técnico; y deterioro de la experiencia del usuario interno, quien no tiene visibilidad sobre el estado de sus pedidos.

ServiDesk se posiciona en este espacio del mercado. No pretende competir con las soluciones empresariales globales en cobertura de funcionalidades; pretende ofrecer exactamente lo que el segmento de empresas medianas latinoamericanas necesita: una herramienta accesible, en español, con un modelo de pricing adaptado a la realidad económica regional, y con la simplicidad operativa suficiente para ser adoptada sin un proceso de implementación complejo.

2. Justificación de la metodología

El equipo ha seleccionado Scrum como marco de trabajo para el desarrollo del proyecto. Esta decisión se fundamenta en tres factores principales: la naturaleza incremental del producto, que se beneficia de entregas funcionales frecuentes; la composición del equipo, conformado por seis estudiantes que requieren una estructura de coordinación clara y ceremonias predecibles; y el cronograma académico, que se alinea naturalmente con sprints quincenales sumando un total de siete iteraciones a lo largo del semestre.

Adicionalmente, Scrum permite gestionar de manera ordenada los cambios que surgirán inevitablemente durante el desarrollo. El Product Backlog se mantiene como artefacto vivo, refinado al final de cada sprint, lo que habilita al equipo a incorporar nuevos requisitos o ajustes sin romper el ritmo de entrega. Esta flexibilidad es particularmente valiosa en un contexto donde el producto se construye desde cero y se espera que tanto el alcance como las decisiones técnicas evolucionen a medida que el equipo gana experiencia con las herramientas seleccionadas.

II. ANÁLISIS DEL PRODUCTO

1. Antecedentes y problemática del sector

El mercado latinoamericano de gestión de servicios TI (ITSM, IT Service Management) está dominado por soluciones globales como Zendesk, Freshdesk, ServiceNow y Jira Service Management, cuyos precios por agente oscilan entre USD 19 y USD 150 mensuales según el plan contratado. Estos costos, sumados a la complejidad de implementación y a la necesidad de personal capacitado para administrar las plataformas, dejan fuera del mercado a un segmento significativo: las empresas medianas que cuentan con un área TI interna pero no disponen del presupuesto ni la infraestructura organizacional para adoptar estas soluciones.

Como consecuencia, el panorama operativo de la pequeña y mediana empresa latinoamericana en cuanto a soporte TI presenta tres patrones consistentes:

- Gestión informal por canales no especializados: WhatsApp, correo electrónico personal o conversación directa son los canales predominantes para reportar incidencias técnicas. La información queda dispersa, no es buscable y se pierde con el tiempo.
- Ausencia de indicadores de desempeño: sin un sistema que registre tiempos de respuesta, tiempos de resolución, tickets atendidos por técnico o categorías recurrentes de incidencias, la gestión del área TI opera sobre percepciones subjetivas en lugar de datos.
- Falta de visibilidad para el usuario final: el empleado que reporta una incidencia no tiene forma de saber si su pedido fue recibido, quién lo atenderá, cuál es la prioridad asignada o en qué momento puede esperar una respuesta.

Esta situación no es accidental, sino estructural. El problema no es que los líderes de TI desconozcan las soluciones existentes; es que el costo-beneficio de adoptar una herramienta empresarial diseñada para empresas Fortune 500 no se justifica en una organización de 50 empleados. ServiDesk surge precisamente como respuesta a esa brecha del mercado: una plataforma diseñada desde el inicio para el segmento intermedio, con un precio que se ajusta a la realidad regional y con la simplicidad operativa que permite implementarla sin un proyecto de transformación digital de fondo.

2. Análisis de la competencia

Para fundamentar la propuesta de valor de ServiDesk, se realizó un análisis comparativo de las principales soluciones del mercado:

Aspecto	Zendesk	Freshdesk	ServiceNow	ServiDesk
Precio inicial mensual	USD 19/agente	USD 15/agente	USD 100+/usuario	S/.0 (plan Gratis)
Idioma del soporte	Multilingüe	Multilingüe	Multilingüe	Español nativo
Pasarela de pago local	No	No	No	Culqi (Perú)
Complejidad de adopción	Media	Baja-media	Alta	Baja
IA integrada	Sí (premium)	Sí (premium)	Sí	Sí (plan Pro)
Plan totalmente gratuito	No	Limitado	No	Sí (50 tickets/mes)

El cuadro evidencia los espacios donde ServiDesk se diferencia: pricing accesible, integración con métodos de pago locales, simplicidad operativa, y un plan totalmente gratuito que permite a las empresas probar el producto sin riesgo antes de migrar a un plan pagado.

3. Stakeholders e interesados

Los stakeholders del producto se identifican por rol funcional dentro de las empresas objetivo. Cada uno tiene necesidades específicas que la plataforma debe atender:

Rol	Necesidades principales	Dolor actual (sin ServiDesk)
Administrador TI	Visibilidad del desempeño del equipo, métricas de tiempo de respuesta, control de carga laboral, identificación de incidencias recurrentes.	Gestiona el área a ciegas, sin métricas objetivas. No puede justificar contrataciones ni inversiones en herramientas con datos duros.
Técnico de soporte	Recibir tickets organizados con prioridad clara, historial de soluciones previas, capacidad de documentar lo que se hizo.	Recibe pedidos desordenados por múltiples canales, pierde tiempo buscando información que ya resolvió antes, atiende emergencias sin contexto.
Usuario final	Reportar incidencias de forma simple, saber el estado de su pedido, recibir notificaciones de avance, evaluar la atención recibida.	No sabe si su mensaje fue leído, no tiene forma de hacer seguimiento, depende de insistir personalmente para ser atendido.
Gerencia / Tomador de decisión	Justificación de inversión en herramientas TI, cálculo de ROI, comparación con alternativas, reportes ejecutivos.	No tiene visibilidad del costo real del soporte ineficiente, no puede dimensionar el problema, opera bajo intuiciones.

4. Alcance del proyecto

El alcance del proyecto se define en términos de qué se incluye dentro del entregable del semestre y qué queda explícitamente fuera. Esta delimitación es crítica para mantener el foco del equipo y garantizar que las funcionalidades core lleguen a estar plenamente implementadas y probadas.

Incluido en el alcance

- Sistema de autenticación multi-tenant con roles diferenciados (administrador, técnico, usuario final).
- Aislamiento completo de datos entre empresas clientes mediante arquitectura multi-tenancy.
- Módulo de gestión de tickets: creación, asignación, cambio de estado, comentarios, adjuntos básicos.
- Sistema de notificaciones por correo electrónico para eventos relevantes (ticket creado, asignado, resuelto).
- Módulo de inteligencia artificial básica: clasificación automática de tickets por categoría y sugerencias de solución, usando la API de Google Gemini.
- Dashboard de analytics con gráficos de tickets por categoría, tiempos de respuesta, tiempos de resolución y rendimiento por técnico.
- Sistema de evaluación de tickets cerrados mediante calificación de uno a cinco estrellas y comentario opcional.
- Base de conocimiento con artículos categorizados, búsqueda full-text y vinculación con tickets resueltos.
- Tres planes de suscripción configurados (Gratis, Básico, Pro) con pasarela de pago real integrada (Culqi en modo Test).

- Despliegue funcional en producción mediante infraestructura serverless (Vercel + Neon).

Fuera del alcance

- Módulo de soporte remoto (control remoto de equipos tipo TeamViewer): se considera fuera del alcance por la complejidad de infraestructura que requiere y se proyecta como funcionalidad de roadmap futuro.
- Facturación electrónica con cumplimiento SUNAT: ServiDesk genera comprobantes internos vía Culqi, pero no emite facturas formales electrónicas.
- Integraciones nativas con herramientas externas de ITSM (Jira, Asana, Slack): si se requieren, se manejan mediante exportación CSV manual.
- Aplicación móvil nativa: la plataforma se entrega como aplicación web responsiva, accesible desde cualquier navegador móvil sin necesidad de instalación.
- Modelos de IA entrenados internamente: la clasificación y sugerencias usan la API de Google Gemini; no se entrenan modelos propietarios.

5. Objetivos

5.1. Objetivo general

Desarrollar ServiDesk, una plataforma SaaS de gestión de soporte técnico TI que permita a empresas medianas administrar tickets, asignar técnicos, generar reportes analíticos y acceder a módulos avanzados como inteligencia artificial básica, bajo un modelo de suscripción mensual escalable que se ajusta al presupuesto y la realidad operativa del mercado latinoamericano.

5.2. Objetivos específicos

- Implementar un módulo de gestión de tickets que cubra el ciclo completo: creación por parte del usuario final, clasificación automática asistida por IA, asignación manual o automática a un técnico, seguimiento mediante estados, resolución y evaluación final.
- Desarrollar un módulo de IA básica que utilice la API de Google Gemini para clasificar tickets entrantes en categorías predefinidas (hardware, software, red, accesos, otros) y sugerir soluciones basadas en tickets históricos resueltos.
- Construir un dashboard de analytics con visualizaciones de tickets por categoría, distribución de tiempos de respuesta, distribución de tiempos de resolución, rendimiento individual de técnicos y tendencias temporales.
- Integrar un sistema de evaluación posterior a la resolución del ticket, permitiendo al usuario calificar la atención recibida con una escala de uno a cinco estrellas y dejar un comentario opcional.
- Implementar una base de conocimiento navegable y buscable, donde los administradores puedan publicar artículos de auto-solución que los usuarios consulten antes de crear un ticket.
- Configurar tres planes de suscripción diferenciados con límites de uso aplicados a nivel de aplicación y backend, y procesamiento de pagos recurrentes vía Culqi en modo Test.
- Desplegar el sistema en producción sobre infraestructura serverless gratuita (Vercel + Neon) con un dominio público funcional y certificado HTTPS automático.

III. REQUISITOS DEL PRODUCTO

1. Requisitos funcionales

Los requisitos funcionales describen las capacidades específicas que el sistema debe ofrecer. Se han categorizado por módulo y se identifican con un código único (RF-XX) para trazabilidad a lo largo del proyecto.

Código	Módulo	Requisito
RF-01	Autenticación	El sistema debe permitir el registro de nuevas empresas (tenants) con un usuario administrador inicial.
RF-02	Autenticación	El sistema debe permitir el inicio de sesión mediante correo y contraseña, así como el inicio de sesión con cuentas de Google (OAuth).
RF-03	Autenticación	El sistema debe diferenciar tres roles de usuario: administrador, técnico y usuario final, cada uno con permisos específicos sobre las funcionalidades del sistema.
RF-04	Multi-tenant	El sistema debe garantizar el aislamiento absoluto de datos entre empresas: ningún usuario de una empresa puede acceder a datos de otra bajo ninguna circunstancia.
RF-05	Tickets	El sistema debe permitir a usuarios finales crear tickets con título, descripción, categoría sugerida y prioridad inicial.
RF-06	Tickets	El sistema debe permitir asignar tickets a técnicos disponibles, ya sea manualmente por el administrador o automáticamente según reglas de distribución.
RF-07	Tickets	El sistema debe gestionar estados del ticket: Abierto, En Progreso, En Espera, Resuelto, Cerrado, con transiciones controladas.
RF-08	Tickets	El sistema debe permitir agregar comentarios al ticket por parte de cualquier participante (creador, técnico asignado, administrador).
RF-09	Notificaciones	El sistema debe enviar correos electrónicos automáticos al usuario final cuando: su ticket es creado, asignado a un técnico, cambia de estado o es resuelto.
RF-10	Notificaciones	El sistema debe enviar un correo al técnico cuando se le asigna un nuevo ticket.
RF-11	IA	El sistema debe clasificar automáticamente cada ticket entrante en una categoría usando la API de Google Gemini (disponible solo en plan Pro).
RF-12	IA	El sistema debe sugerir hasta tres posibles soluciones al técnico asignado, basándose en el contenido del ticket y artículos de la base de conocimiento (plan Pro).
RF-13	Analytics	El sistema debe mostrar un dashboard con gráficos de tickets por categoría, tiempo promedio de respuesta, tiempo promedio de resolución y rendimiento individual de técnicos.
RF-14	Evaluación	El sistema debe solicitar al usuario final una evaluación de uno a cinco estrellas al cerrar un ticket, permitiendo un comentario opcional.
RF-15	Base de conocimiento	El sistema debe permitir a administradores crear, editar y publicar artículos categorizados en la base de conocimiento.
RF-16	Base de conocimiento	El sistema debe ofrecer búsqueda full-text sobre los artículos de la base de conocimiento, accesible para todos los roles.

RF-1 7	Suscripciones	El sistema debe ofrecer tres planes de suscripción: Gratis, Básico y Pro, con límites de uso aplicados a nivel de aplicación.
RF-1 8	Suscripciones	El sistema debe integrar la pasarela de pago Culqi (modo Test) para procesar los pagos recurrentes mensuales de los planes Básico y Pro.
RF-1 9	Suscripciones	El sistema debe procesar los webhooks de Culqi para actualizar el estado de las suscripciones en función de los eventos de pago.
RF-2 0	Suscripciones	El sistema debe bloquear funcionalidades específicas según el plan activo: por ejemplo, restringir el uso de IA al plan Pro o limitar la creación de tickets a 50 mensuales en el plan Gratis.

2. Requisitos no funcionales

Código	Categoría	Requisito
RNF-01	Usabilidad	La interfaz debe ser completamente responsiva y funcionar correctamente en navegadores modernos (Chrome, Firefox, Safari, Edge) y en dispositivos móviles sin necesidad de instalación.
RNF-02	Usabilidad	La interfaz debe estar completamente en español, sin elementos sin traducir, incluyendo mensajes de error y notificaciones.
RNF-03	Rendimiento	Las páginas principales deben cargar en menos de 2 segundos en una conexión de banda ancha estándar (10 Mbps).
RNF-04	Rendimiento	Las consultas a la base de datos deben optimizarse mediante índices apropiados y paginación en listados largos.
RNF-05	Disponibilidad	El sistema debe tener una disponibilidad objetivo del 99% mensual, apoyándose en la infraestructura serverless de Vercel.
RNF-06	Escalabilidad	La arquitectura debe soportar al menos 100 empresas registradas simultáneamente sin degradación perceptible del rendimiento.
RNF-07	Mantenibilidad	El código fuente debe seguir convenciones consistentes verificadas mediante ESLint y Prettier, y debe estar versionado en GitHub.
RNF-08	Mantenibilidad	Las migraciones de la base de datos deben gestionarse mediante Prisma Migrate, asegurando reversibilidad y trazabilidad.

3. Requisitos de seguridad

La seguridad del sistema se aborda mediante un enfoque de defensa en profundidad, donde múltiples capas se complementan para proteger los datos de las empresas clientes.

Código	Categoría	Requisito
RS-01	Autenticación	La autenticación se gestiona mediante Clerk, que implementa estándares OAuth 2.0 y OpenID Connect, con tokens JWT firmados y rotados automáticamente.
RS-02	Almacenamiento de credenciales	Las contraseñas nunca se almacenan en la base de datos propia. Clerk se encarga del hasheo y almacenamiento, utilizando algoritmos modernos como Argon2id.
RS-03	Multi-tenancy	El aislamiento de datos se garantiza mediante dos capas: filtrado obligatorio por organizationId en todas las consultas (capa de aplicación) y políticas de Row-Level Security en PostgreSQL (capa de base de datos).
RS-04	Comunicaciones	Todo el tráfico entre el navegador del usuario y los servidores se realiza sobre HTTPS, con certificados TLS gestionados automáticamente por Vercel.
RS-05	Datos de tarjeta	La información de tarjeta de crédito nunca toca los servidores de ServiDesk. Culqi procesa el tokenizado en el lado del cliente y solo recibimos un token de un solo uso, cumpliendo la normativa PCI DSS 3.2.
RS-06	Autorización	Cada endpoint del backend valida el rol del usuario y su pertenencia a la organización antes de procesar cualquier operación.
RS-07	Webhooks	Los webhooks de Culqi se validan mediante firma criptográfica para garantizar que provienen efectivamente del proveedor y no de un atacante.

RS-08	Secretos	Las claves API (Clerk, Culqi, Gemini, base de datos) se gestionan mediante variables de entorno y nunca se incluyen en el repositorio público.
--------------	----------	--

4. Product Backlog inicial

El Product Backlog se ha organizado en ocho épicas que agrupan funcionalidades relacionadas. Cada épica contiene historias de usuario que serán refinadas y estimadas al inicio de cada sprint.

4.1. Épicas del producto

ID	Épica	Descripción	Prioridad
E1	Autenticación y multi-tenant	Registro de empresas, inicio de sesión, gestión de roles, aislamiento de datos.	Must
E2	Gestión de tickets	Creación, asignación, cambio de estado, comentarios y ciclo completo de atención.	Must
E3	Notificaciones por correo	Envío automático de correos en eventos clave (creación, asignación, resolución).	Must
E4	IA básica con Gemini	Clasificación automática de tickets y sugerencias de solución basadas en IA.	Should
E5	Dashboard de analytics	Visualizaciones gráficas de métricas operativas del soporte TI.	Should
E6	Sistema de evaluación	Calificación post-cierre del ticket por parte del usuario final.	Should
E7	Base de conocimiento	Repositorio de artículos categorizados con búsqueda para auto-solución.	Should
E8	Planes y suscripciones	Tres planes diferenciados, integración con Culqi, gestión de límites.	Must

4.2. Historias de usuario por épica

Cada historia de usuario sigue el formato estándar 'Como [rol] quiero [acción] para [beneficio]' e incluye criterios de aceptación específicos que deben cumplirse para considerarla completa.

Épica E1 — Autenticación y multi-tenant

HU-01	Como responsable de TI de una empresa quiero registrar mi empresa en ServiDesk creando una organización con mi cuenta como administrador para comenzar a gestionar el soporte técnico de mi organización.
Criterios de aceptación	- El sistema permite ingresar nombre de la empresa, correo del administrador y contraseña. - Al completar el registro, se crea una organización en Clerk y un registro de empresa en la base de datos. - El administrador queda automáticamente autenticado tras el registro. - Se envía un correo de bienvenida al administrador con el enlace al dashboard.
Estimación	5 story points Prioridad: Alta
HU-02	Como usuario registrado quiero iniciar sesión con mi correo y contraseña, o con mi cuenta de Google para acceder al sistema de forma segura sin recordar muchas credenciales.

Criterios de aceptación	• El sistema permite ingreso con correo y contraseña. • El sistema ofrece inicio de sesión con Google (OAuth 2.0). • Si las credenciales son incorrectas, se muestra un mensaje claro sin revelar si el correo existe. • Tras el login, el usuario es redirigido al dashboard correspondiente a su rol.
Estimación	3 story points Prioridad: Alta
HU-03	Como administrador de una empresa quiero invitar a nuevos usuarios a mi organización con un rol específico para incorporar a mi equipo técnico y a los usuarios finales al sistema.
Criterios de aceptación	• El administrador puede ingresar el correo del invitado y seleccionar su rol (técnico o usuario final). • Se envía una invitación por correo con un enlace de aceptación de 7 días de validez. • Al aceptar, el invitado completa su contraseña y queda asociado a la organización. • El administrador puede ver el estado de las invitaciones (pendiente, aceptada, expirada).
Estimación	5 story points Prioridad: Alta
HU-04	Como administrador del sistema quiero que los datos de mi empresa estén completamente aislados de los de otras empresas para garantizar la confidencialidad de la información operativa.
Criterios de aceptación	• Ningún usuario puede acceder a datos de otra empresa, ni vía interfaz ni vía API directa. • Las consultas a base de datos filtran obligatoriamente por organizationId. • Las políticas de Row-Level Security en PostgreSQL bloquean accesos cruzados aun si el filtro de aplicación falla. • Una prueba de seguridad valida que un usuario de la empresa A no puede leer datos de la empresa B usando herramientas como Postman.
Estimación	8 story points Prioridad: Alta

Épica E2 — Gestión de tickets

HU-05	Como usuario final quiero crear un ticket describiendo mi incidencia técnica para que el área TI registre y atienda mi pedido con trazabilidad.
Criterios de aceptación	• Se solicita título, descripción detallada, categoría sugerida y prioridad. • Al guardar, el ticket recibe un ID único y queda en estado 'Abierto'. • El usuario recibe confirmación en pantalla y por correo. • El ticket aparece en el listado del administrador y en la cola de asignación.
Estimación	5 story points Prioridad: Alta
HU-06	Como administrador quiero asignar tickets a técnicos específicos para distribuir la carga de trabajo y garantizar que cada incidencia tenga un responsable.
Criterios de aceptación	• El administrador ve un selector de técnicos disponibles al abrir un ticket sin asignar. • Al asignar, el ticket cambia a estado 'En Progreso' y el técnico recibe un correo. • El sistema registra la fecha y hora de la asignación. • El administrador puede reasignar el ticket si es necesario.

Estimación	5 story points Prioridad: Alta
HU-07	Como técnico de soporte quiero ver el listado de tickets que me han asignado para gestionar mi cola de trabajo de manera ordenada por prioridad.
Criterios de aceptación	• El técnico ve solo los tickets que le han sido asignados. • La lista se puede filtrar por estado y ordenar por prioridad o fecha. • Cada ticket muestra título, prioridad, estado actual y tiempo transcurrido desde la creación. • Hace clic en un ticket para acceder a su detalle completo.
Estimación	3 story points Prioridad: Alta
HU-08	Como técnico de soporte quiero actualizar el estado del ticket y agregar comentarios sobre el progreso para mantener informado al usuario y dejar trazabilidad de mi trabajo.
Criterios de aceptación	• Los estados disponibles son: Abierto, En Progreso, En Espera, Resuelto, Cerrado. • Las transiciones de estado están controladas (no se puede saltar de 'Abierto' a 'Cerrado' directamente). • Cada cambio de estado genera una entrada en el historial del ticket con fecha, hora y autor. • Los comentarios soportan texto enriquecido básico (negritas, listas, enlaces).
Estimación	5 story points Prioridad: Alta

Épica E3 — Notificaciones por correo

HU-09	Como usuario final quiero recibir correos automáticos sobre el estado de mi ticket para estar al tanto del avance sin necesidad de hacer seguimiento manual.
Criterios de aceptación	• Se envía un correo al crear el ticket confirmando su recepción. • Se envía un correo al asignarlo a un técnico indicando quién lo atenderá. • Se envía un correo al cambiar a 'Resuelto' invitando a evaluar la atención. • Los correos incluyen un enlace directo al ticket en el sistema.
Estimación	3 story points Prioridad: Alta

HU-10	Como técnico de soporte quiero recibir un correo cuando se me asigne un nuevo ticket para no tener que revisar manualmente el sistema para detectar trabajo nuevo.
Criterios de aceptación	El correo incluye el título, prioridad y enlace al ticket. El correo se envía inmediatamente al momento de la asignación. El técnico puede acceder al ticket desde el enlace sin necesidad de iniciar sesión nuevamente si tiene una sesión activa.
Estimación	2 story points Prioridad: Media

Épica E4 — IA básica con Gemini

HU-11	Como administrador de una empresa con plan Pro quiero que el sistema clasifique automáticamente los tickets entrantes en categorías para reducir el tiempo de revisión manual y mejorar la distribución de trabajo.
Criterios de aceptación	• Al crear un ticket, el sistema llama a la API de Gemini con el título y descripción. • La IA devuelve una categoría sugerida entre: hardware, software, red, accesos, otros. • La categoría sugerida se aplica al ticket pero puede ser modificada manualmente. • La funcionalidad solo está disponible para empresas con suscripción activa al plan Pro.
Estimación	5 story points Prioridad: Media

HU-12	Como técnico de soporte en una empresa Pro quiero ver sugerencias de solución generadas por IA al abrir un ticket para acelerar la resolución apoyándome en conocimiento previo.
Criterios de aceptación	• Al abrir un ticket asignado, aparece un panel lateral con hasta 3 sugerencias de solución. • Las sugerencias se generan considerando el contenido del ticket y artículos relacionados de la base de conocimiento. • El técnico puede marcar una sugerencia como útil o no útil para entrenar la calidad del sistema en el futuro. • Si no hay API disponible o se excede el límite gratuito de Gemini, el panel muestra un mensaje informativo.
Estimación	8 story points Prioridad: Media

Épica E5 — Dashboard de analytics

HU-13	Como administrador quiero ver un dashboard con métricas operativas del soporte para tomar decisiones basadas en datos sobre la gestión del área TI.
Criterios de aceptación	• El dashboard muestra: total de tickets del período, tickets por estado, tickets por categoría. • Incluye gráficos: barras (tickets por técnico), líneas (tendencia temporal), torta (distribución por categoría). • Permite filtrar por rango de fechas (últimos 7, 30, 90 días, o personalizado). • Los datos se actualizan en tiempo real al refrescar la página.
Estimación	8 story points Prioridad: Media

HU-14	Como administrador quiero ver el rendimiento individual de cada técnico para identificar cargas desequilibradas y reconocer al equipo.
Criterios de aceptación	Se muestra una tabla con: tickets atendidos, tiempo promedio de resolución, calificación promedio. Las columnas son ordenables. Cada técnico tiene un perfil expandible con su histórico completo de tickets.
Estimación	5 story points Prioridad: Baja

Épica E6 — Sistema de evaluación

HU-15	Como usuario final quiero calificar la atención recibida al cierre del ticket para expresar mi satisfacción y dar retroalimentación al equipo.
Criterios de aceptación	• Al resolver el ticket, se envía un correo invitando a evaluar. • La evaluación es de 1 a 5 estrellas y permite un comentario opcional. • Solo el creador del ticket puede evaluarlo. • Una vez enviada, la evaluación queda registrada y no puede modificarse.
Estimación	3 story points Prioridad: Media

Épica E7 — Base de conocimiento

HU-16	Como administrador quiero crear artículos en la base de conocimiento con texto enriquecido para documentar soluciones recurrentes y reducir la creación de tickets repetidos.
Criterios de aceptación	• El editor permite formato básico (encabezados, listas, negritas, enlaces, imágenes). • Los artículos se categorizan al guardarlos. • Pueden estar en estado 'Borrador' o 'Publicado'. • Solo los artículos publicados son visibles para usuarios finales.
Estimación	5 story points Prioridad: Media

HU-17	Como usuario final quiero buscar en la base de conocimiento antes de crear un ticket para resolver mi problema por mi cuenta si ya hay una solución documentada.
--------------	--

Criterios de aceptación	• Se ofrece un buscador con búsqueda full-text en título y contenido del artículo. • Los resultados se ordenan por relevancia. • Al abrir un artículo, hay un botón 'Esto resolvió mi problema' que cierra el flujo de creación de ticket. • Si el artículo no resolvió el problema, hay un botón 'Crear ticket' que pre-llena el formulario con el contexto del artículo.
Estimación	5 story points Prioridad: Media

Épica E8 — Planes y suscripciones

HU-18	Como administrador quiero ver los planes disponibles y elegir uno que se ajuste a mi empresa para decidir si quiero contratar funcionalidades premium.
Criterios de aceptación	• Se muestra una página con los tres planes (Gratis, Básico S/.29/mes, Pro S/.79/mes) y sus diferencias. • Cada plan tiene un botón 'Suscribirse' que inicia el flujo de pago. • Si el administrador ya tiene un plan activo, se indica claramente cuál es y si quiere cambiarlo.
Estimación	3 story points Prioridad: Alta
HU-19	Como administrador quiero suscribirme a un plan pagado mediante mi tarjeta de crédito para habilitar las funcionalidades premium para mi empresa.
Criterios de aceptación	• El formulario de pago utiliza el checkout seguro de Culqi (modo Test). • Los datos de tarjeta nunca llegan a los servidores de ServiDesk; se tokenizan en el cliente. • Al completar el pago, la suscripción queda activa inmediatamente y se redirige al dashboard. • Se envía un correo de confirmación con el detalle de la suscripción.
Estimación	8 story points Prioridad: Alta
HU-20	Como administrador quiero ver el estado actual de mi suscripción y descargar mis comprobantes para tener visibilidad financiera sobre el servicio contratado.
Criterios de aceptación	• Se muestra el plan activo, fecha del próximo cobro, monto y método de pago. • Se lista el historial de cobros realizados. • Es posible cancelar la suscripción en cualquier momento (la cancelación es efectiva al final del período pagado).
Estimación	5 story points Prioridad: Media
HU-21	Como sistema quiero procesar automáticamente los webhooks de Culqi para mantener el estado de suscripciones actualizado para que el sistema refleje en tiempo real los pagos exitosos o fallidos.
Criterios de aceptación	• Existe un endpoint público que recibe los webhooks de Culqi. • Los webhooks se validan mediante firma criptográfica antes de procesarse. • Los eventos soportados incluyen: cobro exitoso, cobro fallido, suscripción cancelada. • Cada evento actualiza el estado de la suscripción y, si corresponde, notifica al administrador por correo.

5. Definición de Listo (Definition of Ready)

Una historia de usuario se considera lista para entrar al Sprint Backlog cuando cumple los siguientes criterios:

- Tiene un título y descripción claros en formato 'Como [rol] quiero [acción] para [beneficio]'.
- Cuenta con al menos tres criterios de aceptación específicos y verificables.
- Ha sido estimada en story points por el equipo mediante Planning Poker.
- Tiene una prioridad asignada (Alta, Media, Baja) según el método MoSCoW.
- No tiene bloqueos técnicos pendientes ni decisiones de diseño sin resolver.
- Si involucra interfaz, existe un mockup o referencia visual en Stitch.

6. Definición de Terminado (Definition of Done)

Una historia de usuario se considera terminada cuando cumple todos los siguientes criterios:

- El código está implementado, revisado mediante Pull Request y mergeado a la rama main.
- Pasa los chequeos automáticos de ESLint, Prettier y TypeScript sin errores.
- La compilación del proyecto (npm run build) finaliza exitosamente.
- La funcionalidad está desplegada en el entorno de preview de Vercel y verificada manualmente.
- Todos los criterios de aceptación de la historia se cumplen y han sido probados.
- La documentación técnica relevante (README, comentarios en código complejo) está actualizada.
- No introduce regresiones en funcionalidades previas.

IV. EQUIPO Y ORGANIZACIÓN

1. Composición del equipo

El equipo de desarrollo está conformado por seis estudiantes de la Escuela Profesional de Ingeniería de Sistemas e Informática. Cada miembro cuenta con responsabilidades específicas alineadas a su perfil técnico y a los requerimientos del producto.

Integrante	Rol formal	Responsabilidades principales
Danny Steve Ordoñez Navarro	Líder de Proyecto / Product Owner	Coordinación general del equipo, gestión del Product Backlog, definición de la visión del producto, arquitectura general del sistema y supervisión de la integración.
Ludwing Silva Ramos	Desarrollador Backend Senior	Implementación de la API REST, lógica de negocio del módulo de tickets, integración con la API de Google Gemini para el módulo de IA.
Renzo Bereca Noriega	Desarrollador Backend / DBA	Diseño e implementación de la base de datos multi-tenant, configuración de Row-Level Security, integración con Clerk para autenticación, integración con Culqi para suscripciones.
Jhor Anthony Grandez Sifuentes	Desarrollador Frontend Senior	Implementación del dashboard de analytics, componentes interactivos del frontend, integración de gráficos con Recharts, consumo de la API REST.
André de Jesús Burga Saldaña	Desarrollador Frontend / UX/UI	Diseño de las interfaces de usuario en Stitch, definición del sistema de diseño, implementación de componentes UI responsivos, prototipado de flujos.
Alexander Ali Paredes Burga	Scrum Master / Tester / Documentador	Facilitación de las ceremonias Scrum, pruebas funcionales del sistema, redacción de la documentación técnica y del manual de usuario.

2. Roles Scrum

Sobre la estructura del equipo se mapean los tres roles fundamentales de Scrum:

Product Owner

El Product Owner es responsable de maximizar el valor del producto. Define qué se construye y en qué orden, mantiene el Product Backlog actualizado y priorizado, y representa la voz del mercado objetivo dentro del equipo. La priorización del backlog se fundamenta en investigación de mercado, análisis competitivo (Zendesk, Freshdesk, GLPI) y necesidades inferidas del segmento de empresas medianas TI en Latinoamérica.

Asignado a: *Danny Steve Ordoñez Navarro*.

Scrum Master

El Scrum Master es responsable de que el equipo aplique correctamente el marco Scrum. Facilita las ceremonias (Sprint Planning, Daily, Review, Retrospective), elimina los bloqueos que detiene el avance del equipo y protege al equipo de interferencias externas que afecten el sprint en curso.

Asignado a: *Alexander Ali Paredes Burga*

Development Team

El Development Team es responsable de entregar el incremento funcional al final de cada sprint. Está conformado por todos los integrantes del equipo que escriben código y/o ejecutan pruebas: *Ludwing Silva, Alexander Paredes, Jhor Grandez, André Burga y Renzo Bereca.*

3. Matriz de responsabilidades (RACI)

La matriz RACI clarifica quién es Responsable, quién es Aprobador, a quién se Consulta y a quién se Informa por cada área del proyecto.

Área	<i>Danny</i>	<i>Ludwing</i>	<i>Alexander</i>	<i>Jhor</i>	<i>André</i>	<i>Renzo</i>
Arquitectura del sistema	R/A	C	C	I	I	I
API REST e IA	A	R	C	I	I	C
Base de datos y seguridad	A	C	R	I	I	C
Frontend y dashboard	A	I	I	R	C	C
Diseño UX/UI	A	I	I	C	R	I
Pruebas y QA	A	C	C	C	C	R
Documentación	A	C	C	C	C	R
Coordinación Scrum	C	I	I	I	I	R/A

R = Responsable de ejecutar | A = Aprobador final | C = Consultado | I = Informado

V. ARQUITECTURA DEL SISTEMA

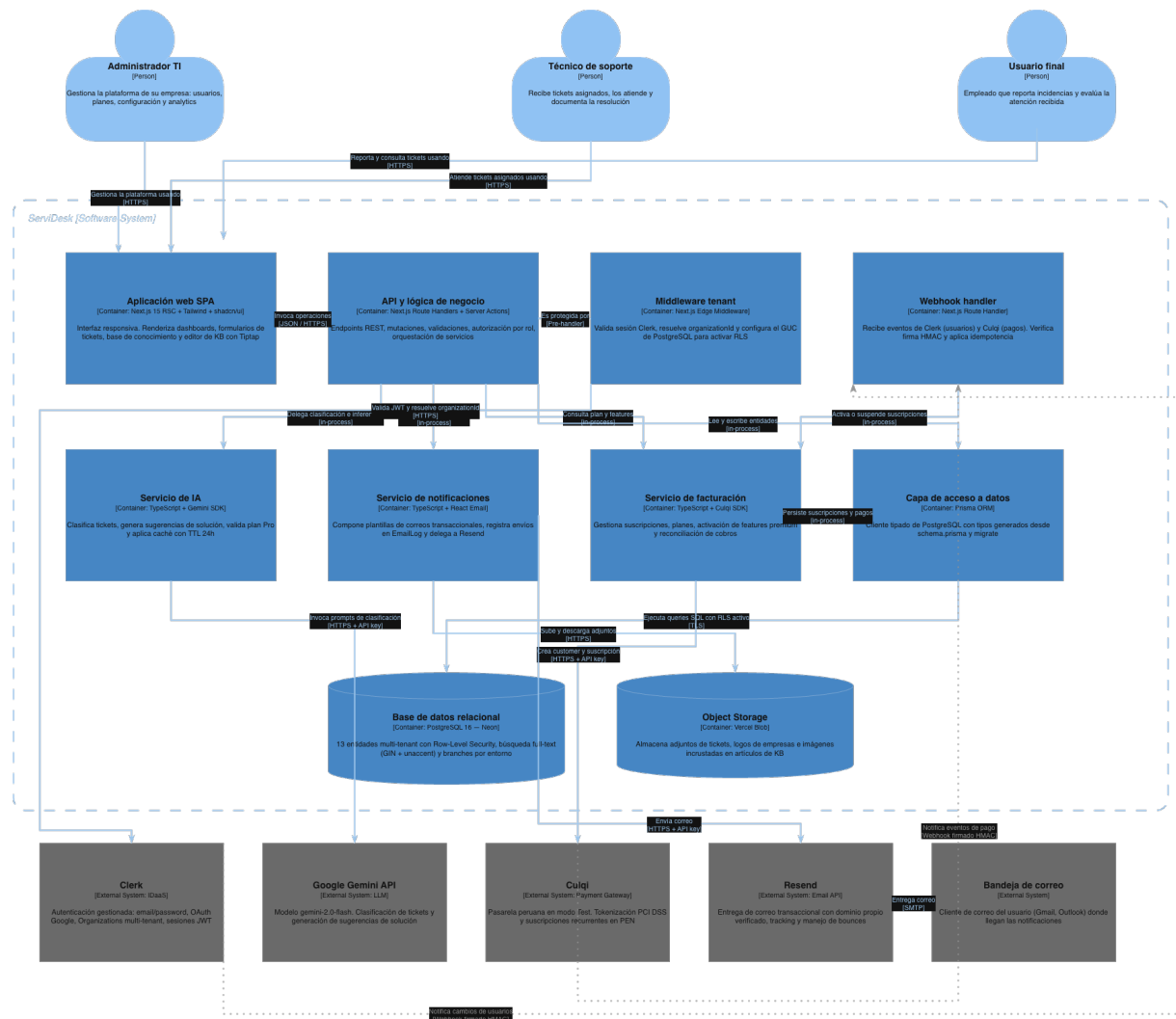
1. Visión general de la arquitectura

ServiDesk se construye sobre una arquitectura serverless de tipo full-stack monolítico modular, donde el frontend y el backend conviven en un único repositorio basado en Next.js 15 con su patrón de App Router. Esta decisión simplifica el desarrollo (un solo lenguaje, un solo deploy, un solo dominio) sin sacrificar capacidad de escalamiento, dado que Vercel gestiona automáticamente el aprovisionamiento de funciones serverless según la carga real.

La arquitectura se compone de cinco capas claramente diferenciadas que interactúan entre sí mediante interfaces bien definidas:

- **Capa de presentación:** componentes React Server Components y Client Components renderizados desde Next.js, estilizados con Tailwind CSS y compuestos a partir de la biblioteca shadcn/ui.
- **Capa de aplicación:** Route Handlers y Server Actions de Next.js que implementan la lógica de negocio, validan permisos y orquestan las llamadas a servicios externos.
- **Capa de datos:** PostgreSQL alojado en Neon, accedido a través de Prisma como ORM tipado, con políticas de Row-Level Security para reforzar el aislamiento multi-tenant.
- **Capa de identidad:** Clerk como proveedor externo de autenticación e identidad, incluyendo Clerk Organizations para la gestión multi-tenant.
- **Capa de servicios externos:** integraciones con Google Gemini API (clasificación e IA), Culqi (suscripciones y pagos) y Resend o similar (envío de correos transaccionales).

Página 24 de 39



3. Stack tecnológico justificado

Capa	Tecnología	Justificación
Framework	Next.js 15 (App Router) + TypeScript	Permite construir frontend y backend en un solo proyecto con tipado fuerte de extremo a extremo. App Router habilita Server Components, que reducen el JavaScript enviado al cliente y mejoran el rendimiento percibido.
Estilos	Tailwind CSS	Framework utility-first que permite construir interfaces rápidamente sin saltar entre archivos CSS. La consistencia visual emerge naturalmente del sistema de diseño embebido.
Componentes UI	shadcn/ui	Biblioteca de componentes accesibles, copy-paste, sin lock-in. Garantiza cumplimiento de WAI-ARIA y se integra perfectamente con Tailwind.
Base de datos	PostgreSQL (Neon)	Base relacional madura con soporte nativo para Row-Level Security, búsquedas full-text y JSON. Neon ofrece un tier gratuito con branching de bases de datos, ideal para entornos de desarrollo y producción.
ORM	Prisma	Genera tipos TypeScript a partir del schema, lo que elimina una clase entera de bugs en consultas. Las migraciones son reversibles y trazables en Git.
Autenticación	Clerk + Clerk Organizations	Plataforma Identity-as-a-Service que delega la complejidad de autenticación, gestión de sesiones, OAuth, magic links y multi-tenancy. Cumple SOC 2 Type II y GDPR.
IA	Google Gemini API (Flash)	Modelo multimodal de alta calidad con un tier gratuito de 1500 requests/día, suficiente para los volúmenes esperados durante el desarrollo y demo del proyecto.
Pasarela de pago	Culqi (modo Test)	Pasarela peruana líder con soporte nativo para suscripciones recurrentes. Modo Test gratuito y sin necesidad de RUC. Cumplimiento PCI DSS 3.2 nativo.
Hosting	Vercel	Plataforma optimizada para Next.js con CDN global, HTTPS automático, deploys atómicos y rollback instantáneo. Tier gratuito generoso.
Diseño UI	Stitch (Google Labs)	Herramienta de diseño asistido por IA que genera mockups de alta fidelidad a partir de descripciones en lenguaje natural y exporta a código Tailwind/React.
Asistencia de desarrollo	Claude Code	Asistente de programación con capacidades agénticas que acelera la implementación, especialmente útil para equipos pequeños o estudiantes con restricciones de tiempo.
Versionado	Git + GitHub	Estándar de la industria. GitHub Projects se utiliza adicionalmente como tablero Scrum.

4. Arquitectura multi-tenant

ServiDesk implementa una arquitectura multi-tenant de tipo Shared Database con identificador de tenant (también conocida como pool model). Esta estrategia comparte una única base de datos PostgreSQL entre todas las empresas clientes, pero garantiza el aislamiento estricto de los datos mediante dos mecanismos complementarios que operan en niveles distintos.

Capa 1: filtrado en consultas (aplicación)

Todas las tablas que contienen datos específicos de una empresa incluyen una columna `organizationId` que referencia a la organización en Clerk. Cada consulta realizada desde la aplicación filtra obligatoriamente por este campo, utilizando el `organizationId` obtenido de la sesión activa de Clerk. Esta capa es la primera línea de defensa y opera a nivel de Prisma.

Capa 2: Row-Level Security (base de datos)

Como segunda capa de protección, PostgreSQL aplica políticas de Row-Level Security sobre todas las tablas multi-tenant. La sesión de la base de datos se configura con el `organizationId` del usuario actual mediante un parámetro de sesión, y las políticas RLS validan que ninguna fila pueda leerse o modificarse si su `organizationId` no coincide. Esto significa que aun si un desarrollador olvidara incluir el filtro en una consulta, la base de datos rechazaría la operación.

Este enfoque de defensa en profundidad es el estándar adoptado por productos SaaS modernos como Linear, Vercel y Notion, y representa un equilibrio óptimo entre simplicidad operativa (una sola base de datos a administrar) y robustez de seguridad.

5. Modelo de despliegue

El despliegue se ejecuta de manera automatizada mediante la integración entre GitHub y Vercel:

- Cada push a una rama feature genera un Preview Deployment con URL única, útil para code reviews y QA.
- Cada merge a la rama main desencadena un deploy a producción (`servidesk.vercel.app`) con cero downtime.
- Las migraciones de base de datos se ejecutan mediante un script post-build que aplica los cambios pendientes.
- Las variables de entorno (claves API de Clerk, Culqi, Gemini, etc.) se gestionan en el panel de Vercel y se inyectan en cada deploy.
- Los logs de aplicación y los errores se centralizan en el panel de Vercel para diagnóstico rápido.

planStatus	Enum (active, past_due, canceled)	Estado del plan según los webhooks de Culqi.
culqiCustomerId	String?	Identificador del cliente en Culqi (para suscripciones recurrentes).
createdAt / updatedAt	DateTime	Timestamps automáticos.

2.2. User (Usuario del sistema)

Representa a cada persona dentro de una empresa. Como la autenticación se delega a Clerk, los datos sensibles (contraseña, correo verificado, sesiones) viven en Clerk. La tabla User local sirve para vincular el usuario de Clerk con datos específicos de ServiDesk (rol, asignaciones, evaluaciones, etc.).

Atributo	Tipo	Descripción
id	String (cuid)	Identificador interno.
clerkUserId	String @unique	Identificador del usuario en Clerk.
organizationId	String (FK)	Empresa a la que pertenece el usuario.
email	String	Correo del usuario (sincronizado desde Clerk).

name	String	Nombre completo.
role	Enum (admin, technician, end_user)	Rol dentro de la organización.
isActive	Boolean	Si el usuario puede iniciar sesión.
createdAt / updatedAt	DateTime	Timestamps automáticos.

2.3. Ticket (Incidencia de soporte)

Entidad central del sistema. Representa cada incidencia o solicitud reportada por un usuario final. Mantiene relaciones con el creador, el técnico asignado, la categoría y los comentarios.

Atributo	Tipo	Descripción
id	String (cuid)	Identificador interno.
organizationId	String (FK)	Empresa propietaria del ticket.
ticketNumber	Int @unique	Número correlativo legible (#001, #002...).
title	String	Título corto de la incidencia.
description	Text	Descripción detallada.
status	Enum	OPEN, IN_PROGRESS, ON_HOLD, RESOLVED, CLOSED.
priority	Enum	LOW, MEDIUM, HIGH, URGENT.

categoryId	String (FK)	Categoría (asignada manualmente o por IA).
createdById	String (FK)	Usuario que creó el ticket.
assignedToId	String? (FK)	Técnico asignado (puede ser nulo).
aiCategorySuggested	String?	Categoría sugerida por la IA en el momento de creación.
resolvedAt	DateTime?	Fecha de resolución (para métricas de tiempo).
closedAt	DateTime?	Fecha de cierre definitivo.
createdAt / updatedAt	DateTime	Timestamps automáticos.

2.4. Entidades complementarias

El modelo completo incluye las siguientes entidades adicionales. Sus atributos específicos se documentan en el archivo schema.prisma del repositorio.

Entidad	Propósito
Category	Categorías de tickets (hardware, software, red, accesos, otros). Configurables por empresa.
Comment	Comentarios dentro de un ticket. Cada uno tiene autor, fecha y contenido en texto enriquecido.
Attachment	Archivos adjuntos a tickets o comentarios. Se almacenan en un storage externo (Vercel Blob).
TicketHistory	Registro inmutable de cambios de estado, asignaciones y eventos del ticket. Sirve como audit trail.
Evaluation	Calificación del ticket por el usuario final (1-5 estrellas) y comentario opcional. Una por ticket.
KnowledgeArticle	Artículo de la base de conocimiento, con título, contenido enriquecido, categoría y estado (borrador/publicado).
SubscriptionPlan	Catálogo de planes disponibles (Gratis, Básico, Pro). Incluye precio, features habilitados y límites.
Subscription	Suscripción activa de una empresa a un plan. Incluye fecha de inicio, próximo cobro y estado de Culqi.
Payment	Historial de cobros realizados sobre una suscripción. Incluye monto, fecha, estado y referencia de Culqi.
EmailLog	Registro de correos transaccionales enviados, para auditoría y diagnóstico.

3. Relaciones principales

- Una Organization tiene muchos Users, Tickets, Categories, KnowledgeArticles y una Subscription activa.
- Un User pertenece a una Organization y puede ser creador de muchos Tickets o asignado a muchos Tickets.

- Un Ticket pertenece a una Category, tiene un creador (User), opcionalmente un técnico asignado (User), muchos Comments y una Evaluation.
- Una Subscription referencia a un SubscriptionPlan y tiene muchos Payments.
- Todas las entidades multi-tenant están relacionadas con Organization mediante organizationId (FK con ON DELETE CASCADE).

4. Índices y optimizaciones

Para garantizar el rendimiento del sistema desde el primer día, se definen los siguientes índices en la base de datos:

- Índice compuesto en (organizationId, status) sobre Ticket: acelera los filtros por estado dentro de una empresa.
- Índice compuesto en (organizationId, assignedToId) sobre Ticket: acelera la cola de tickets de cada técnico.
- Índice en createdAt sobre Ticket: necesario para los gráficos de tendencias temporales en el dashboard.
- Índice GIN sobre los campos title y content de KnowledgeArticle: habilita búsqueda full-text eficiente.
- Índice único en (organizationId, ticketNumber) sobre Ticket: garantiza la numeración correlativa única por empresa.

5. Políticas de Row-Level Security

Las políticas RLS se aplican a todas las tablas multi-tenant. La política general establece que un registro solo puede ser leído o modificado si su organizationId coincide con el valor del parámetro de sesión app.current_org_id, que la aplicación configura al inicio de cada request usando el organizationId obtenido de Clerk.

VII. Diseño de interfaces

1. Sistema de diseño

El sistema de diseño de ServiDesk se construye sobre los componentes de shadcn/ui y se personaliza mediante los tokens de diseño de Tailwind. La paleta cromática prioriza la legibilidad, el contraste accesible (WCAG AA mínimo) y la coherencia visual entre las distintas secciones del producto.

Paleta cromática

Rol	Hex	Token Tailwind	Uso
Primario	#1F4E79	primary	Botones principales, enlaces, navegación.
Primario claro	#D9E2F3	primary/10	Fondos sutiles, hover states, etiquetas informativas.
Éxito	#16A34A	green-600	Estados resueltos, confirmaciones, evaluaciones positivas.
Advertencia	#F59E0B	amber-500	Estados en espera, alertas no críticas.
Error / Urgente	#DC2626	red-600	Errores, prioridad urgente, acciones destructivas.
Texto principal	#111827	gray-900	Cuerpo de texto, titulares.
Texto secundario	#6B7280	gray-500	Subtítulos, leyendas, metadata.
Fondo neutro	#F9FAFB	gray-50	Fondo de la aplicación, separadores de secciones.

Tipografía

Se utiliza Inter como tipografía principal por su legibilidad en pantalla y su soporte completo de pesos. Inter es gratuita, open source y está disponible en Google Fonts.

Estilo	Tamaño	Peso	Uso
Display	36px	700	Títulos principales de página.
Heading 1	28px	600	Encabezados de sección.
Heading 2	22px	600	Subtítulos.
Body Large	16px	400	Cuerpo de texto principal.
Body	14px	400	Listas, tablas, formularios.
Caption	12px	500	Metadata, etiquetas, leyendas.

2. Mapa de navegación (sitemap)

La estructura general de navegación del sistema se organiza por rol de usuario. Cada rol accede a un conjunto específico de pantallas, y la barra lateral de navegación se adapta en consecuencia.

Estructura de pantallas por rol

Pantalla	Roles con acceso
Landing pública	Visitantes (sin autenticar)
Registro de empresa	Visitantes
Inicio de sesión	Visitantes
Dashboard principal	Administrador (vista completa), Técnico (vista de su carga), Usuario final (vista de sus tickets)
Listado de tickets	Todos los roles (con filtros distintos según el rol)
Detalle de ticket	Todos los roles (con permisos diferenciados)
Crear ticket	Usuario final, Técnico (en nombre de un usuario)
Base de conocimiento (listado y búsqueda)	Todos los roles
Artículo de base de conocimiento (detalle)	Todos los roles (edición solo Admin)
Gestión de usuarios	Administrador
Gestión de categorías	Administrador
Planes y suscripción	Administrador
Configuración de la organización	Administrador
Perfil personal	Todos los roles

3. Flujos de usuario principales

A continuación se documentan los tres flujos críticos del sistema, que representan los caminos principales que los usuarios recorren durante su uso cotidiano.

Flujo 1: Creación y resolución de un ticket

Este es el flujo central del producto y debe ser óptimo en términos de fricción para el usuario final.

- 1. El usuario final inicia sesión y accede al dashboard.

- 2. Hace clic en 'Crear ticket' y completa el formulario (título, descripción, categoría sugerida, prioridad).
- 3. Antes de guardar, el sistema sugiere artículos de la base de conocimiento que podrían resolver su problema.
- 4. Si ninguno resuelve, confirma la creación del ticket.
- 5. El sistema (plan Pro) clasifica automáticamente la categoría usando Gemini.
- 6. El administrador (o regla automática) asigna el ticket a un técnico.
- 7. El técnico recibe correo de notificación y atiende el ticket. Cambia el estado a 'En Progreso'.
- 8. El técnico documenta su intervención con comentarios y finalmente cambia el estado a 'Resuelto'.
- 9. El usuario final recibe correo de invitación a evaluar y cierra el ticket con su calificación.
- 10. La métrica se incorpora al dashboard de analytics del administrador.

Flujo 2: Suscripción a un plan pagado

- 1. El administrador navega a 'Planes y suscripción' desde el menú lateral.
- 2. Compara los tres planes (Gratis, Básico, Pro) y selecciona uno.
- 3. Hace clic en 'Suscribirme' y es llevado al checkout de Culqi.
- 4. Ingresar los datos de su tarjeta (los datos se tokenizan en el navegador, no llegan a ServiDesk).
- 5. Culqi procesa el pago y notifica a ServiDesk vía webhook.
- 6. La suscripción queda activa y el administrador es redirigido al dashboard con un mensaje de éxito.
- 7. Las funcionalidades premium se desbloquean inmediatamente para todos los usuarios de la empresa.

Flujo 3: Consulta de la base de conocimiento antes de crear ticket

- 1. El usuario final ingresa al sistema y va a 'Base de conocimiento'.
- 2. Busca su problema mediante el buscador o navega por categorías.
- 3. Encuentra un artículo relevante y lo abre.
- 4. Lee el contenido y aplica la solución sugerida.
- 5. Marca el artículo como 'Esto resolvió mi problema' o, si no funcionó, hace clic en 'Crear ticket' (el cual pre-llena el contexto del artículo consultado).

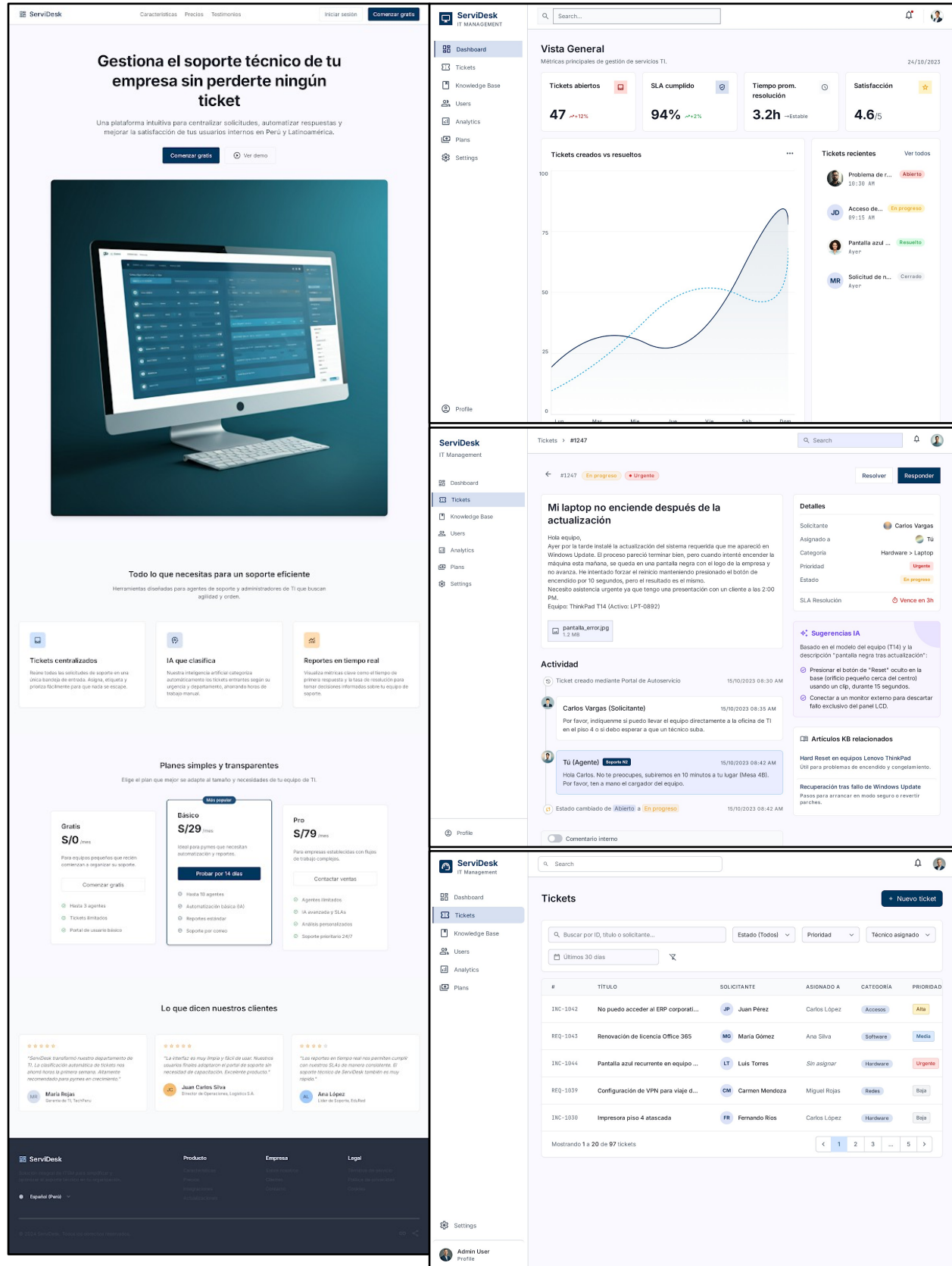
4. Pantallas a diseñar en Stitch

Las siguientes pantallas conforman el conjunto mínimo a diseñar para el avance actual. Cada una debe contar con versión desktop y versión móvil. Stitch genera ambas versiones automáticamente al solicitarlo.

ID	Pantalla	Elementos clave a incluir
P-0 1	Landing pública	Hero con propuesta de valor, comparativa de planes, testimonios (placeholder), llamado a la acción 'Crear cuenta gratis'.
P-0 2	Registro de empresa	Formulario: nombre de la empresa, correo del administrador, contraseña, aceptación de términos. Opción de registrarse con Google.
P-0 3	Inicio de sesión	Formulario de correo + contraseña, botón de Google, enlace de '¿Olvidaste tu contraseña?'.
P-0 4	Dashboard del administrador	Métricas principales (tickets abiertos, resueltos hoy, tiempo promedio), gráfico de tendencia, listado de tickets recientes, acceso rápido a acciones comunes.
P-0 5	Dashboard del técnico	Cola de tickets asignados al técnico, ordenados por prioridad. Indicador de tickets urgentes. Botón rápido para cambiar estado.
P-0 6	Dashboard del usuario final	Listado de sus propios tickets, botón principal 'Crear ticket', acceso a base de conocimiento.
P-0 7	Crear ticket	Formulario: título, descripción enriquecida, categoría, prioridad, archivos adjuntos. Panel lateral con artículos de KB relacionados (búsqueda en tiempo real).
P-0 8	Detalle de ticket	Cabecera con título, ID, estado, prioridad. Cuerpo con descripción, historial de comentarios. Panel lateral con metadata (creador, técnico, fechas) y, si es plan Pro, sugerencias de IA.
P-0 9	Listado de tickets	Tabla paginada con filtros (estado, prioridad, categoría, técnico, rango de fechas), búsqueda por texto, ordenamiento por columnas.
P-1 0	Base de conocimiento (listado)	Buscador prominente, navegación por categorías, tarjetas con preview de artículos más vistos.
P-1 1	Artículo de base de conocimiento	Contenido enriquecido del artículo, sidebar con artículos relacionados, botones '¿Resolvió tu problema?' (Sí / No → crear ticket).
P-1 2	Editor de artículo de KB	Editor de texto enriquecido (estilo Notion), selector de categoría, estado (Borrador / Publicado), preview en tiempo real.
P-1 3	Gestión de usuarios	Tabla de usuarios con rol, estado, fecha de ingreso. Botón 'Invitar usuario'. Acciones: cambiar rol, desactivar, eliminar.
P-1 4	Modal de invitar usuario	Campo de correo, selector de rol, mensaje personalizado opcional. Botón 'Enviar invitación'.
P-1 5	Planes y suscripción	Tres tarjetas comparativas (Gratis, Básico, Pro) con features, precio y botón 'Suscribirme'. Sección inferior con plan actual y estado.
P-1 6	Checkout de Culqi	Esta pantalla es proporcionada por Culqi mediante su widget embebido. En el diseño solo se documenta la integración visual.

P-1 7	Analytics avanzado	Múltiples widgets gráficos: tickets por categoría (torta), tiempo de resolución (líneas), rendimiento por técnico (barras horizontales), distribución de evaluaciones.
P-1 8	Configuración de la organización	Nombre de la empresa, logo, dominio, gestión de categorías personalizadas, configuración de notificaciones.
P-1 9	Perfil personal	Datos del usuario (gestionados por Clerk), preferencias de notificación, cambio de contraseña.
P-2 0	Estado vacío y errores	Pantallas para casos especiales: 'Aún no tienes tickets', '404 - Página no encontrada', '403 - No tienes permiso', 'Error de conexión'.

5. Mockups de alta fidelidad



6. Prototipo navegable

Las pantallas diseñadas se conectan entre sí mediante el sistema de prototipado interactivo de Stitch, generando un flujo navegable que permite recorrer la aplicación como si fuese real, antes de iniciar el desarrollo. Esto es especialmente útil para validar la usabilidad con stakeholders y detectar problemas de flujo antes de escribir código.

Prototipo disponible en: <https://stitch.withgoogle.com/projects/5844956485543268642>

7. Consideraciones de diseño responsivo

Todas las pantallas están diseñadas siguiendo un enfoque mobile-first y se adaptan a tres breakpoints principales:

- **Móvil (< 768px):** una sola columna, navegación inferior tipo bottom-nav o menú hamburguesa, tablas convertidas en tarjetas apiladas.
- **Tablet (768px - 1024px):** dos columnas en algunas vistas, navegación lateral colapsable, tablas con scroll horizontal cuando es necesario.
- **Desktop (> 1024px):** layout completo de tres zonas (navegación lateral, contenido principal, panel contextual), tablas completas con todas las columnas visibles.

8. Accesibilidad

El diseño cumple con los siguientes principios de accesibilidad, garantizando que la plataforma sea utilizable por personas con diferentes capacidades:

- Contraste mínimo de 4.5:1 entre texto y fondo (WCAG AA).
- Todos los elementos interactivos tienen un área de toque mínima de 44x44 píxeles.
- Los formularios incluyen etiquetas semánticas (<label>) asociadas a sus campos.
- Los estados (foco, hover, error) son claramente visibles y no dependen únicamente del color.
- La navegación es completamente posible mediante teclado.
- Los componentes de shadcn/ui que se utilizan cumplen los patrones WAI-ARIA por defecto.

Próximos pasos

Con la finalización del presente documento, el equipo cierra la fase de análisis y diseño del proyecto. Los próximos entregables del semestre cubrirán las siguientes etapas:

Sprint	Semanas	Objetivo del sprint
Sprint 0	1 - 2	Análisis, diseño de interfaces y setup técnico. Corresponde al presente entregable.
Sprint 1	3 - 4	Implementación de autenticación con Clerk, configuración de multi-tenancy con Row-Level Security en PostgreSQL.
Sprint 2	5 - 6	Módulo de gestión de tickets completo: creación, asignación, estados, comentarios. Integración del sistema de notificaciones por correo.
Sprint 3	7 - 8	Integración de Google Gemini API para clasificación automática de tickets y generación de sugerencias de solución.
Sprint 4	9 - 10	Dashboard de analytics con visualizaciones gráficas. Sistema de evaluación post-cierre del ticket.
Sprint 5	11 - 12	Base de conocimiento completa con búsqueda full-text. Integración de Culqi y planes de suscripción funcionales.
Sprint 6	13 - 14	Integración final, pruebas exhaustivas, corrección de bugs, documentación de usuario, preparación de la presentación final.